

EML — hybrid formal/informal report

Auto-formalization of arXiv:2603.21852 (Odrzywółek)

2026-05-01

Contents

EML formalization — hybrid report	2
Status dashboard	2
Index	2
001_def_eml Definition of the EML operator	7
002_def_eml_term Inductive type of EML terms	7
003_def_eml_eval Evaluation of EML terms	8
004_def_edl EDL variant (Exp Divided by Log)	8
005_def_neg_eml Negated-EML variant	9
006_eml_one_one_eq_e $\text{eml}(1,1) = e$	9
007_eml_x_one_eq_exp $\text{eml}(x,1) = \exp(x)$	9
008_eml_one_y $\text{eml}(1,y) = e - \ln(y)$	10
009_eml_x_e $\text{eml}(x, e) = \exp(x) - 1$	10
010_eml_pos_left Positivity of the left exponential of eml	11
011_ln_via_eml Natural logarithm via EML	11
012_exp_via_eml $\exp(x)$ as eml — corollary phrasing of 007	11
013_sub_via_eml Subtraction expressed via EML	12
014_add_via_eml Addition via Identity 1 (Exp-Log reduction)	12
015_mul_via_exp_log Multiplication via Identity 1 (Exp-Log reduction)	12
016_add_via_exp_log Additive consequence: $x + y$ via exp and ln (specialized)	13
017_successor_negation_identity Successor / negation identity	13
018_inv_successor_inv_inverse_simple Algebraic simplification of $\text{inv}(\text{suc}(\text{inv } x))$	14
019_negation_in_calc3 Negation realised in the Calc-3 set	14
020_emlterm_size Size function on EML terms	14
021_emlterm_size_pos EML term size is positive	15
022_emlterm_e_witness An EML term whose eval is e	15
023_emlterm_exp_x_witness EML term with x-leaf whose eval is $\exp(x)$	16
024_wolfram_to_calc3 · Wolfram $\rightarrow$ Calc 3 reduction	16
025_calc3_to_calc2 · Calc 3 $\rightarrow$ Calc 2 reduction	17
026_calc2_to_calc1 · Calc 2 $\rightarrow$ Calc 1 reduction	17
027_calc1_to_calc0 · Calc 1 $\rightarrow$ Calc 0 reduction	18
028_calc0_to_eml · Calc 0 $\rightarrow$ EML reduction	18
029_eml_minimality · Minimality: three primitives is the minimum	18
030_emlterm_for_zero EMLTerm whose eval is 0	19
031_emlterm_for_neg_one EMLTerm whose eval is -1	19

032_amlterm_for_two	EMLTerm whose eval is 2	20
033_amlterm_for_half	EMLTerm whose eval is 1/2	21
034_amlterm_for_pi	· EMLTerm whose eval is	23
035_amlterm_for_i	· EMLTerm whose eval is i (imaginary unit)	24
036_amlterm_for_neg_x	EMLTerm realising the function $-x$	24
037_amlterm_for_inv_x	EMLTerm realising $1/x$ (for $x > 0$)	26
038_amlterm_for_sq_x	EMLTerm realising x^2 (for $x > 0$)	28
039_amlterm_for_sqrt_x	· EMLTerm realising the function $\sqrt{x}$	30
040_amlterm_for_add_xy	EMLTerm realising $x + y$	30
041_amlterm_for_mul_xy	EMLTerm realising $x \cdot y$	31
042_amlterm_for_pow_xy	EMLTerm realising x^y (for $0 < x$ and $0 < y$)	32
043_master_formula_param_count	· Master-formula parameter count at level n	35
044_amlterm_count_catalan	· Count of EMLTerms equals the Catalan number	35
045_main_completeness_stub	· Main completeness theorem — stub	36

EML formalization — hybrid report

This document interleaves the paper *All elementary functions from a single binary operator* (arXiv:2603.21852, A. Odrzywołek) with the corresponding Lean 4 + Mathlib v4.28 artifacts. Proofs were produced by Aristotle (Harmonic) plus hand-curated definitions.

Status dashboard

Status	Count	Symbol
Verified	32	
Partial	1	
Submitted	0	...
Failed	0	
Pending	12	·
Total	45	

Index

ID	Title	Kind	Diff	Section
001_def_aml	Definition of the EML operator	definition	1	§3 Results, Equation 3
002_def_aml	Inductive type of EML terms	definition	1	§4.2 Elementary functions as binary trees
003_def_aml	Evaluation of EML terms	definition	1	§4.1 EML compiler

ID	Title	Kind	Diff	Section
004_def_edl	EDL variant (Exp Divided by Log)	definition	1	§3 Results, Identity 4b
005_def_neg_Negated-	Negated- EML variant	definition	1	§3 Results, Identity 4c
006_eml_one_eml(1, d) = e	$\text{eml}(1, d) = e$	identity	1	§3 Results, EML expression catalog
007_eml_x_oneml(x, 1)exp	$\text{oneml}(x, 1)\text{exp}$ $\exp(x)$	identity	1	§3 Results, EML expression catalog
008_eml_one_eml(1, y) = e - ln(y)	$\text{eml}(1, y) = e - \ln(y)$	identity	1	§3 Results (conse- quence of Equation 3)
009_eml_x_e	$\text{eml}(x, e) = \exp(x) - 1$	identity	1	§3 Results (conse- quence of Equation 3)
010_eml_pos_Positivity of the left exponential of eml	Positivity of the left exponential of eml	theorem	1	§3 Results (implicit; pre- condition lemma)
011_ln_via_eml	Natural logarithm via EML	identity	3	§3 Results, Identity 5
012_exp_via_exp(x) as eml — corollary phrasing of 007	$\exp(x)$ as eml — corollary phrasing of 007	identity	2	§3 Results, EML expression catalog
013_sub_via_Subtraction expressed via EML	Subtraction expressed via EML	identity	2	§3 Results, Calculator- equivalence chain
014_add_via_Addition via Identity 1 (Exp-Log reduction)	Addition via Identity 1 (Exp-Log reduction)	identity	2	§3 Results, Identity 1

ID	Title	Kind	Diff	Section
015_mul_via_Multiplication	Identity 1 (Exp-Log reduction)	identity	2	§3 Results, Identity 1
016_add_via_Addition	consequence: $x + y$ via $\exp$ and $\ln$ (specialized)	identity	2	§3 Results, Identity 1 (specialised consequence)
017_successor_Succession	negation identity	identity	2	§3 Results (passing remark)
018_inv_successor_Algebraic	inversion simplification of $\text{inv}(\text{succ}(\text{inv } x))$	identity	1	§3 Results (sub-step of successor identity)
019_negation_Negation	realised in the Calc-3 set	calculator- equivalence	3	§3 Results, Table 2 row 'Calc 3'
020_amlterm_Size	function on EML terms	definition	2	§4.1 EML compiler (‘K denotes the size of the RPN code’)
021_amlterm_EML_pos	size is positive	theorem	2	§4.1 EML compiler (implicit)
022_amlterm_AnyEML	term whose eval is e	theorem	2	§3 Results, EML expression catalog (e, K=3)
023_amlterm_EML_x	term with x-leaf whose eval is $\exp(x)$	theorem	3	§3 Results, EML expression catalog ($\exp(x)$, K=3)

	ID	Title	Kind	Diff	Section
·	024_wolfram_	Wolfram 3 → Calc 3 reduction	calculator- equivalence	4	§3 Results, Table 2 (rows 'Wolfram' and 'Calc 3')
·	025_calc3_to_	Calc3 → Calc 2 reduction	calculator- equivalence	3	§3 Results, Table 2 (rows 'Calc 3' and 'Calc 2')
·	026_calc2_to_	Calc2 → Calc 1 reduction	calculator- equivalence	3	§3 Results, Table 2 (rows 'Calc 2' and 'Calc 1')
·	027_calc1_to_	Calc1 → Calc 0 reduction	calculator- equivalence	3	§3 Results, Table 2 (rows 'Calc 1' and 'Calc 0')
·	028_calc0_to_	Calc0 → EML reduction	calculator- equivalence	4	§3 Results, Table 2 (rows 'Calc 0' and 'EML')
·	029_eml_minimality:	Minimality: three primitives is the minimum	theorem	5	§3 Results (concluding remark on Table 2)
	030_emlterm_	EMLTerm whose eval is 0	theorem	4	§3 Results, EML expression catalog (0, K=7)
	031_emlterm_	EMLTerm whose eval is -1	theorem	4	§3 Results, EML expression catalog (-1, K=17)
	032_emlterm_	EMLTerm whose eval is 2	theorem	4	§3 Results, EML expression catalog (2, K=27)

ID	Title	Kind	Diff	Section
033_amlterm_amlTerm	EMLTerm whose eval is 1/2	theorem	4	§3 Results, EML expression catalog (1/2, K=91)
034_amlterm_amlTerm	EMLTerm whose eval is	theorem	5	§3 Results, EML expression catalog (, K=193)
035_amlterm_amlTerm	EMLTerm whose eval is i (imaginary unit)	theorem	5	§3 Results, EML expression catalog (i, K=131)
036_amlterm_amlTermx	EMLTerm realising the function $-x$	theorem	5	§3 Results, EML expression catalog ($-x$, K=57)
037_amlterm_amlTermx	EMLTerm realising $1/x$ (for $x > 0$)	theorem	5	§3 Results, EML expression catalog ($1/x$, K=65)
038_amlterm_amlTermx	EMLTerm realising x^2 (for $x > 0$)	theorem	5	§3 Results, EML expression catalog (x^2 , K=75)
039_amlterm_amlTermx	EMLTerm realising the function $\sqrt{x}$	theorem	5	§3 Results, EML expression catalog ($\sqrt{x}$, K=139)
040_amlterm_amlTermxy	EMLTerm realising x $+ y$	theorem	5	§3 Results, EML expression catalog ($x +$ y , K=27)
041_amlterm_amlTermxy	EMLTerm realising $x \cdot$ y	theorem	5	§3 Results, EML expression catalog ($x \times$ y , K=41)

ID	Title	Kind	Diff	Section
042_amlterm_EMLTerm_xy	EMLTerm realising x^y (for $0 < x$ and $0 < y$)	theorem	5	§3 Results, EML expression catalog (x^y , K=49)
043_master_formula_parameter_count_at_level_n	Master-paramdef formula parameter count at level n	definition	2	§4.3 Master formula — symbolic regression
044_amlterm_Count_of_Catalan	Count of EMLTerms equals the Catalan number	theorem	4	§4.2 Elementary functions as binary trees ('Catalan structures')
045_main_completeness_theorem	Minicross completeness theorem — stub	theorem	5	§3 Results, abstract claim of universality

001_def_aml Definition of the EML operator

Paper section: §3 Results, Equation 3 • *Status:* complete • *Difficulty:* 1/5

$$\text{eml}(x, y) = \exp(x) - \ln(y)$$

The EML operator is a binary real-valued operator defined by $\text{eml}(x,y) = \exp(x) - \ln(y)$. It is the heart of Odrzywołek's construction: combined with the constant 1 it generates every elementary function of a scientific calculator.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

/-- The EML (Exp-Minus-Log) binary operator on the reals.
Equation 3 in Odrzywołek (arXiv:2603.21852). -/
def eml (x y : ℝ) : ℝ := Real.exp x - Real.log y

end EML
```

002_def_aml_term Inductive type of EML terms

Paper section: §4.2 Elementary functions as binary trees • *Status:* complete • *Difficulty:* 1/5

$$S \rightarrow 1 \mid \text{eml}(S, S)$$

EML terms are full binary trees: every leaf is the constant 1, and every internal node is an application of the eml operator to two subtrees. The language is isomorphic to the Catalan structures.

```

namespace EML

/-- Constant-only EML term grammar from §4.2:
  `S → 1 | eml(S, S)` -/
inductive EMLTerm : Type
| one : EMLTerm
| eml : EMLTerm → EMLTerm → EMLTerm
deriving Repr

end EML

```

003_def_eml_eval Evaluation of EML terms

Paper section: §4.1 EML compiler • *Status:* complete • *Difficulty:* 1/5

Each EML term evaluates to a real number obtained by replacing every leaf 1 by the constant 1 and every internal node $\text{eml}(t, u)$ by $\exp(\text{eval } t) - \ln(\text{eval } u)$.

The `eval` function maps each EML term to a real number: a `.one` leaf gives 1, and a `.eml t u` node gives $\exp(\text{eval } t) - \ln(\text{eval } u)$. Because `Real.log` is junk-valued at non-positive arguments the function is total on `ℝ`, but the ‘meaningful’ values arise only when the subtrees evaluate to positive reals.

```

import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

inductive EMLTerm : Type
| one : EMLTerm
| eml : EMLTerm → EMLTerm → EMLTerm
deriving Repr

/-- Real-valued evaluation of an EML term. -/
def EMLTerm.eval : EMLTerm → ℝ
| .one => 1
| .eml t u => Real.exp (EMLTerm.eval t) - Real.log (EMLTerm.eval u)

end EML

```

004_def_edl EDL variant (Exp Divided by Log)

Paper section: §3 Results, Identity 4b • *Status:* complete • *Difficulty:* 1/5

$\text{edl}(x, y) = \exp(x) / \ln(y)$, constant: e

The EDL variant uses division instead of subtraction: $\text{edl}(x, y) = \exp(x)/\ln(y)$, with the constant e replacing 1. It has analogous universality properties but a different ‘starting point’.

```

import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

/-- The EDL (Exp Divided by Log) variant of EML.
Identity 4b in Odrzywołek (arXiv:2603.21852). Constant: `e`. -/
def edl (x y : ℝ) : ℝ := Real.exp x / Real.log y

```



```
end EML
```

005_def_neg_eml Negated-EML variant

Paper section: §3 Results, Identity 4c • *Status:* complete • *Difficulty:* 1/5

$$-\text{eml}(y, x) = \ln(x) - \exp(y), \text{ constant: } -\infty$$

The third variant negates EML and swaps the arguments: $-\text{eml}(y, x) = \ln(x) - \exp(y)$. The paper labels its required constant as $-\infty$, reflecting a topological difference when trying to express 1.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

/-- The negated-EML variant:  $-\text{eml}(y, x) = \ln(x) - \exp(y)$ .
Identity 4c in Odrzywołek (arXiv:2603.21852). -/
def negEml (x y : ℝ) : ℝ := Real.log x - Real.exp y

end EML
```

006_eml_one_one_eq_e $\text{eml}(1,1) = e$

Paper section: §3 Results, EML expression catalog • *Status:* complete • *Difficulty:* 1/5

$$e = \text{eml}(1, 1)$$

Applying the EML operator to two unit arguments yields Euler's number: $\text{eml}(1,1) = \exp(1) - \ln(1) = e - 0 = e$. The simplest of the fundamental examples.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

noncomputable def eml (x y : ℝ) : ℝ := Real.exp x - Real.log y

theorem eml_one_one : eml 1 1 = Real.exp 1 := by
  -- By definition of eml, we have eml 1 1 = Real.exp 1 - Real.log 1.
  simp [eml]

end EML
```

007_eml_x_one_eq_exp $\text{eml}(x,1) = \exp(x)$

Paper section: §3 Results, EML expression catalog • *Status:* complete • *Difficulty:* 1/5

$$\exp(x) = \text{eml}(x, 1)$$

Setting the second argument to 1 collapses $\ln(1)$ to zero, so $\text{eml}(x,1) = \exp(x)$ for every real x . This shows that the exponential is immediately reachable in EML.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic
```

```

namespace EML

noncomputable def eml (x y : ℝ) : ℝ := Real.exp x - Real.log y

theorem eml_x_one (x : ℝ) : eml x 1 = Real.exp x := by
  unfold eml; norm_num;

end EML

```

008_eml_one_y $\text{eml}(1, y) = e - \ln(y)$

Paper section: §3 Results (consequence of Equation 3) • *Status:* complete • *Difficulty:* 1/5

$$\text{eml}(1, y) = \exp(1) - \ln(y)$$

Setting the first argument to 1 reduces $\exp(1)$ to the constant e , giving $\text{eml}(1, y) = e - \ln(y)$. The hypothesis $y > 0$ is not formally required (Real.log is junk-valued elsewhere) but we keep it for semantic clarity.

```

import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

noncomputable def eml (x y : ℝ) : ℝ := Real.exp x - Real.log y

theorem eml_one_y (y : ℝ) (hy : 0 < y) : eml 1 y = Real.exp 1 - Real.log y := by
  simp [eml]

end EML

```

009_eml_x_e $\text{eml}(x, e) = \exp(x) - 1$

Paper section: §3 Results (consequence of Equation 3) • *Status:* complete • *Difficulty:* 1/5

$$\text{eml}(x, e) = \exp(x) - \ln(e) = \exp(x) - 1$$

Substituting $y = e$ yields $\ln(e) = 1$, so $\text{eml}(x, e) = \exp(x) - 1$. The value $\exp(x) - 1$ appears throughout analysis (Taylor expansions, etc.) so it is worth isolating.

```

import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

noncomputable def eml (x y : ℝ) : ℝ := Real.exp x - Real.log y

theorem eml_x_e (x : ℝ) : eml x (Real.exp 1) = Real.exp x - 1 := by
  -- By definition of eml, we have eml x (Real.exp 1) = Real.exp x - Real.log (Real.exp 1).
  simp [eml]

end EML

```

010_eml_pos_left Positivity of the left exponential of eml

Paper section: §3 Results (implicit; pre-condition lemma) • *Status:* complete • *Difficulty:* 1/5

$\exp(x) > 0$ for every real x , hence the leading $\exp$ term in $\text{eml}(x,y)$ is always positive.

The left summand of EML, $\exp(x)$, is always strictly positive. This trivial lemma underpins later positivity arguments (e.g. for the $\log$ argument in chunk 011).

```
import Mathlib.Analysis.SpecialFunctions.Exp

namespace EML

theorem eml_left_pos (x y : ℝ) : 0 < Real.exp x := by
  positivity

end EML
```

011_ln_via_eml Natural logarithm via EML

Paper section: §3 Results, Identity 5 • *Status:* complete • *Difficulty:* 3/5

$$\ln(z) = \text{eml}(1, \text{eml}(\text{eml}(1, z), 1))$$

The natural log expands as a triple-nested EML application: $\ln(z) = \text{eml}(1, \text{eml}(\text{eml}(1, z), 1))$. The proof unfolds eml repeatedly and uses Real.log_one , Real.log_exp plus arithmetic. For $z > 0$ all the inner Real.log arguments are positive.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

noncomputable def eml (x y : ℝ) : ℝ := Real.exp x - Real.log y

theorem ln_via_eml (z : ℝ) (hz : 0 < z) :
  Real.log z = eml 1 (eml (eml 1 z) 1) := by
  unfold eml at *;
  norm_num +zetaDelta at *

end EML
```

012_exp_via_eml $\exp(x)$ as eml — corollary phrasing of 007

Paper section: §3 Results, EML expression catalog • *Status:* complete • *Difficulty:* 2/5

$$\exp(x) = \text{eml}(x, 1)$$

A re-statement of chunk 007 in ‘definitional’ direction: $\exp(x)$ is exactly $\text{eml}(x,1)$. The proof typically just inherits via $(\text{eml_x_one } x).\text{symm}$.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML
```

```

noncomputable def eml (x y : ) : := Real.exp x - Real.log y

theorem exp_via_eml (x : ) : Real.exp x = eml x 1 := by
  simp [eml, Real.log_one]

end EML

```

013_sub_via_eml Subtraction expressed via EML

Paper section: §3 Results, Calculator-equivalence chain • *Status:* complete • *Difficulty:* 2/5

$$x - y = \exp(\ln x) - \exp(\ln y) \text{ (when } x, y > 0\text{); equivalently } \exp(\ln x) - \ln(\exp y).$$

The subtraction $x - y$ can be written as $\text{eml}(\ln x, \exp y) = \exp(\ln x) - \ln(\exp y) = x - y$ when $x > 0$ (the $\ln(\exp y) = y$ simplification needs no positivity, but $\exp(\ln x) = x$ needs $x > 0$).

```

import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

noncomputable def eml (x y : ) : := Real.exp x - Real.log y

theorem sub_via_eml (x y : ) (hx : 0 < x) :
  x - y = eml (Real.log x) (Real.exp y) := by
  unfold eml; rw [ Real.exp_log hx, Real.log_exp ] ;

end EML

```

014_add_via_eml Addition via Identity 1 (Exp-Log reduction)

Paper section: §3 Results, Identity 1 • *Status:* complete • *Difficulty:* 2/5

$$x + y = \ln(\exp(x) \times \exp(y))$$

Addition arises from the exp homomorphism: $x + y = \ln(\exp x \cdot \exp y)$. A canonical transcendental identity; in Mathlib it follows from `Real.log_mul` plus `Real.log_exp`.

```

import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

theorem add_via_exp_log (x y : ) :
  x + y = Real.log (Real.exp x * Real.exp y) := by
  rw [ ← Real.exp_add, Real.log_exp ]

end EML

```

015_mul_via_exp_log Multiplication via Identity 1 (Exp-Log reduction)

Paper section: §3 Results, Identity 1 • *Status:* complete • *Difficulty:* 2/5

$$x \times y = \exp(\ln x + \ln y)$$

The product of positive reals is recovered by exponentiating the sum of logs: $x \cdot y = \exp(\ln x + \ln y)$ for $x, y > 0$. The multiplicative half of Identity 1.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

theorem mul_via_exp_log (x y : ℝ) (hx : 0 < x) (hy : 0 < y) :
  x * y = Real.exp (Real.log x + Real.log y) := by
  -- Using the property of logarithms that  $\log(ab) = \log(a) + \log(b)$ , we can rewrite the
  ↪ right-hand side.
  rw [Real.exp_add, Real.exp_log hx, Real.exp_log hy]

end EML
```

016_add_via_exp_log Additive consequence: $x + y$ via exp and ln (specialized)

Paper section: §3 Results, Identity 1 (specialised consequence) • *Status:* complete • *Difficulty:* 2/5

$$x + y = \ln(\exp(x) \cdot \exp(y))$$

A re-statement of chunk 014 split out so that Aristotle can use either (`log_mul` + `log_exp`) or a direct combined lemma. Useful for the calculator-equivalence chain in Group 6.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

theorem add_eq_log_mul_exp (x y : ℝ) :
  x + y = Real.log (Real.exp x) + Real.log (Real.exp y) := by
  rw [Real.log_exp, Real.log_exp]

end EML
```

017_successor_negation_identity Successor / negation identity

Paper section: §3 Results (passing remark) • *Status:* complete • *Difficulty:* 2/5

$$\text{succ}(\text{inv}(\text{pre}(\text{inv}(\text{succ}(\text{inv}(x))))) = 1/(1/(1/x + 1) - 1) + 1 = -x$$

Identity: $1/(1/(1/x + 1) - 1) + 1 = -x$. A single `field_simp`; `ring` should close it; the side conditions are $x \neq 0$ and $x \neq -1$ to keep all denominators alive.

```
import Mathlib

namespace EML

theorem successor_negation_identity (x : ℝ) (hx : x ≠ 0) (hx1 : x ≠ -1) :
  1 / (1 / (1 / x + 1) - 1) + 1 = -x := by
  grind

end EML
```

018_inv_successor_inv_inverse_simple Algebraic simplification of $\text{inv}(\text{suc}(\text{inv } x))$

Paper section: §3 Results (sub-step of successor identity) • *Status:* complete • *Difficulty:* 1/5

$$1/(1/x + 1) = x / (1 + x)$$

Auxiliary algebraic lemma: $1/(1/x + 1) = x/(1+x)$ for $x \neq 0$ and $1+x \neq 0$. Used as an intermediate step in 017 and 019.

```
import Mathlib

namespace EML

theorem inv_successor_inv (x : ) (hx : x ≠ 0) (hx1 : x + 1 ≠ 0) :
  1 / (1 / x + 1) = x / (1 + x) := by
  have h1x : 1 + x ≠ 0 := by rw [add_comm]; exact hx1
  field_simp

end EML
```

019_negation_in_calc3 Negation realised in the Calc-3 set

Paper section: §3 Results, Table 2 row 'Calc 3' • *Status:* complete • *Difficulty:* 3/5

$-x$ is realised in Calc 3 (operators $\{+, \exp, \ln, -x, 1/x\}$) using only $+$, $1/x$ and the standalone $-x$ primitive — but the bootstrap formula in Identity (suc/inv) shows the operation is derivable from $+$ and $1/x$ alone.

We show that negation $-x$ is reachable using only addition and inversion (the Calc-3 operator set minus the standalone $-x$ primitive): $-x = 1/(1/(1/x+1)-1) + 1$ for $x \neq 0$, $x \neq -1$. This is the step that removes $-x$ as a primitive on the way to Calc 2/Calc 1.

```
import Mathlib

namespace EML

theorem neg_via_calc3 (x : ) (hx : x ≠ 0) (hx1 : x ≠ -1) :
  -x = 1 / (1 / (1 / x + 1) - 1) + 1 := by
  grind

end EML
```

020_emlterm_size Size function on EML terms

Paper section: §4.1 EML compiler ('K denotes the size of the RPN code') • *Status:* complete • *Difficulty:* 2/5

K denotes the size of the RPN code (number of EML/leaf nodes in the binary tree).

We define the size of an EML term as the number of nodes: `.one` has size 1, and `.eml t u` has size $1 + \text{size } t + \text{size } u$. This matches the K column ('EML compiler K') in the paper's catalogue.

```
namespace EML
```

```

inductive EMLTerm : Type
| one : EMLTerm
| eml : EMLTerm → EMLTerm → EMLTerm
deriving Repr

/-- Number of nodes in an EML term. -/
def EMLTerm.size : EMLTerm →
| .one => 1
| .eml t u => 1 + EMLTerm.size t + EMLTerm.size u

end EML

```

021_emlterm_size_pos EML term size is positive

Paper section: §4.1 EML compiler (implicit) • *Status:* complete • *Difficulty:* 2/5

Every EML term has at least one node, so $K \geq 1$.

For every EML term, $\text{size } t \geq 1$. Inductive proof: leaves have size 1; nodes have $1 + \text{size } t + \text{size } u \geq 1$.

```

import Mathlib

namespace EML

inductive EMLTerm : Type
| one : EMLTerm
| eml : EMLTerm → EMLTerm → EMLTerm
deriving Repr

def EMLTerm.size : EMLTerm →
| .one => 1
| .eml t u => 1 + EMLTerm.size t + EMLTerm.size u

theorem EMLTerm.size_pos (t : EMLTerm) : 1 ≤ EMLTerm.size t := by
  induction' t using EMLTerm.recOn with t ih <|> norm_num [ EMLTerm.size ];
  grind

end EML

```

022_emlterm_e_witness An EML term whose eval is e

Paper section: §3 Results, EML expression catalog (e, K=3) • *Status:* complete • *Difficulty:* 2/5

e: $\text{eml}(1, 1) \text{ — } K = 3$.

The EML term $\text{eml}(\text{one}, \text{one})$ evaluates to $\exp(1) - \ln(1) = e$. A constructive witness that e lies in the image of eval . The term's size is 3 (one node + two leaves), matching the K column in the paper's catalogue.

```

import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

inductive EMLTerm : Type

```

```

| one : EMLTerm
| eml : EMLTerm → EMLTerm → EMLTerm
deriving Repr

noncomputable def EMLTerm.eval : EMLTerm →
| .one => 1
| .eml t u => Real.exp (EMLTerm.eval t) - Real.log (EMLTerm.eval u)

theorem emlterm_e_witness : EMLTerm.eval (.eml .one .one) = Real.exp 1 := by
simp [EMLTerm.eval, Real.log_one]

end EML

```

023__emlterm__exp__x__witness EML term with x-leaf whose eval is exp(x)

Paper section: §3 Results, EML expression catalog (exp(x), K=3) • *Status:* complete • *Difficulty:* 3/5

exp(x): eml(x, 1) — K = 3.

To realise exp(x) as an EML term we add a .var leaf representing the variable x; the evaluation at x gives EMLTerm.eval x (.eml .var .one) = exp(x). We introduce a separate type EMLTerm to avoid disturbing the original (constants-only) EMLTerm.

```

import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

/-- EML term grammar with a single distinguished variable `x`. -/
inductive EMLTerm : Type
| one : EMLTerm
| var : EMLTerm
| eml : EMLTerm → EMLTerm → EMLTerm
deriving Repr

/-- Evaluation of a parameterised EML term at value `x`. -/
noncomputable def EMLTerm.eval (x : ) : EMLTerm →
| .one => 1
| .var => x
| .eml t u => Real.exp (EMLTerm.eval x t) - Real.log (EMLTerm.eval x u)

theorem emlterm1_exp_x_witness (x : ) :
EMLTerm.eval x (.eml .var .one) = Real.exp x := by
simp [EMLTerm.eval, Real.log_one]

end EML

```

024__wolfram__to__calc3 · Wolfram → Calc 3 reduction

Paper section: §3 Results, Table 2 (rows 'Wolfram' and 'Calc 3') • *Status:* pending • *Difficulty:* 4/5

From the 7-symbol Wolfram set { , e, i, ln, +, ×, } we can drop , e, i and the binary × and , replacing them with {exp, ln, -x, 1/x, +} (Calc 3, 6 symbols).

First step of the reduction chain: every function expressible in the Wolfram set is expressible in Calc 3. We state it as an existential; a constructive proof would require defining the ‘Wolfram language’ and an interpreter, which we defer.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

/-- Placeholder for the Wolfram → Calc 3 reduction (Table 2, row 1 → row 2).
A full statement requires interpreters for both languages; we leave a `True`
witness so the dependency edge is recorded. -/
theorem wolfram_subset_calc3 : True := by
  sorry

end EML
```

025_calc3_to_calc2 · Calc 3 → Calc 2 reduction

Paper section: §3 Results, Table 2 (rows 'Calc 3' and 'Calc 2') • *Status:* pending • *Difficulty:* 3/5

From Calc 3 $\{\exp, \ln, -x, 1/x, +\}$ we drop $-x, 1/x$ and replace $+$ with $-$ to obtain Calc 2 $\{\exp, \ln, -\}$ (4 symbols).

Canonical step: $-x$ is removed via the successor identity (chunk 019); $1/x$ via algebraic identities; $+$ is replaced by $-$ because $a + b = a - (-b)$ and $-b$ is now available. Stated as a placeholder existential.

```
namespace EML

/-- Placeholder for the Calc 3 → Calc 2 reduction (Table 2, row 2 → row 3). -/
theorem calc3_subset_calc2 : True := by
  sorry

end EML
```

026_calc2_to_calc1 · Calc 2 → Calc 1 reduction

Paper section: §3 Results, Table 2 (rows 'Calc 2' and 'Calc 1') • *Status:* pending • *Difficulty:* 3/5

From Calc 2 $\{\exp, \ln, -\}$ we move to Calc 1 $\{e \text{ or } \cdot, x^y, \log_x(y)\}$.

Move from a unary $\{\exp, \ln\}$ + binary $\{-\}$ to two binary primitives $\{x^y, \log_x(y)\}$ together with a constant e (or $\cdot$). Uses $\exp(x) = e^x$ and $\ln(x) = \log_e(x)$.

```
namespace EML

/-- Placeholder for the Calc 2 → Calc 1 reduction (Table 2, row 3 → row 4). -/
theorem calc2_subset_calc1 : True := by
  sorry

end EML
```

027_calc1_to_calc0 · Calc 1 $\rightarrow$ Calc 0 reduction

Paper section: §3 Results, Table 2 (rows 'Calc 1' and 'Calc 0') • *Status:* pending • *Difficulty:* 3/5

From Calc 1 $\{e, x^y, \log_x(y)\}$ we drop the constant and replace x^y with $\exp(x)$, reaching Calc 0 $\{\exp, \log_x(y)\}$ (3 symbols).

Substitute x^y by $\exp(y \cdot \ln x)$, keeping $\exp$ as the only unary, and recover e as $\exp(1)$. Requires the binary $\log_x(y)$ and the unary $\exp$.

```
namespace EML

/-- Placeholder for the Calc 1  $\rightarrow$  Calc 0 reduction (Table 2, row 4  $\rightarrow$  row 5). -/
theorem calc1_subset_calc0 : True := by
  sorry

end EML
```

028_calc0_to_eml · Calc 0 $\rightarrow$ EML reduction

Paper section: §3 Results, Table 2 (rows 'Calc 0' and 'EML') • *Status:* pending • *Difficulty:* 4/5

From Calc 0 $\{\exp, \log_x(y)\}$ we collapse to EML $\{1, \text{eml}(\cdot, \cdot)\}$ — $\exp(x) = \text{eml}(x, 1)$ and $\log_x(y)$ is built from the natural log via Identity 5.

The strongest step of the chain: reduction to $\{1, \text{eml}\}$. Uses both $\exp(x) = \text{eml}(x, 1)$ (chunk 007) and $\ln(z) = \text{eml}(1, \text{eml}(\text{eml}(1, z), 1))$ (chunk 011). Stated as an existential; a constructive proof would need an interpreter.

```
namespace EML

/-- Placeholder for the Calc 0  $\rightarrow$  EML reduction (Table 2, row 5  $\rightarrow$  row 6),
which is the paper's central claim. -/
theorem calc0_subset_eml : True := by
  sorry

end EML
```

029_eml_minimality · Minimality: three primitives is the minimum

Paper section: §3 Results (concluding remark on Table 2) • *Status:* pending • *Difficulty:* 5/5

Three primitives is the minimum: any further reduction would either drop the constant (leaving an unsatisfiable arity equation) or merge eml with another operation in a way that loses expressiveness.

States that no calculator configuration with fewer than three primitives retains full elementary expressiveness. A formal proof requires defining ‘calculator’ and ‘expressible’; we leave a stub.

```
namespace EML

/-- Placeholder for the minimality claim (no calculator with < 3 primitives
remains universal). Open question in the paper; we keep a `True` witness. -/
theorem eml_minimality_stub : True := by
```

```

    sorry
end EML

```

030_amlterm_for_zero EMLTerm whose eval is 0

Paper section: §3 Results, EML expression catalog (0, K=7) • *Status:* complete • *Difficulty:* 4/5

0: $K = 7$ (literal tree in Supplementary).

There exists an EML term of size 7 evaluating to 0. The paper reports $K=7$ but defers the literal tree to the Supplementary; we state existence and leave the witness as `sorry` until the literal tree is transcribed.

```

import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

inductive EMLTerm : Type
| one : EMLTerm
| eml : EMLTerm → EMLTerm → EMLTerm
deriving Repr

noncomputable def EMLTerm.eval : EMLTerm →
| .one => 1
| .eml t u => Real.exp (EMLTerm.eval t) - Real.log (EMLTerm.eval u)

theorem emlterm_for_zero : t : EMLTerm, EMLTerm.eval t = 0 := by
-- Witness: eml one (eml (eml one one) one)
-- eval = exp(1) - log(exp(exp(1) - log(1)))
--       = exp(1) - log(exp(exp(1) - 0))
--       = exp(1) - log(exp(exp(1)))
--       = exp(1) - exp(1) = 0
exact .eml .one (.eml (.eml .one .one) .one), by
  simp [EMLTerm.eval, Real.log_one, sub_zero, Real.log_exp, sub_self]

end EML

```

031_amlterm_for_neg_one EMLTerm whose eval is -1

Paper section: §3 Results, EML expression catalog (-1, K=17) • *Status:* complete • *Difficulty:* 4/5

-1: $K = 17$ (literal tree in Supplementary).

There exists an EML term of size 17 evaluating to -1. Existential; the literal tree is in the Supplementary.

```

import Mathlib

namespace EML

inductive EMLTerm : Type

```

```

| one : EMLTerm
| eml : EMLTerm → EMLTerm → EMLTerm
deriving Repr

noncomputable def EMLTerm.eval : EMLTerm →
| .one => 1
| .eml t u => Real.exp (EMLTerm.eval t) - Real.log (EMLTerm.eval u)

theorem emlterm_for_neg_one : t : EMLTerm, EMLTerm.eval t = -1 := by
-- Let's choose the term $t = .eml (.eml .one (.eml (.eml .one (.eml .one (.eml .one
↪ .one))) .one)) (.eml (.eml .one .one) .one)$.
use .eml (.eml .one (.eml (.eml .one (.eml .one (.eml .one .one))) .one)) (.eml (.eml .one
↪ .one) .one);
-- Let's simplify the expression step by step.
simp [EMLTerm.eval];
rw [Real.exp_log ] <|> linarith [Real.add_one_le_exp 1 ]

end EML

```

032_emlterm_for_two EMLTerm whose eval is 2

Paper section: §3 Results, EML expression catalog (2, K=27) • *Status:* complete • *Difficulty:* 4/5

2: K = 27 (literal tree in Supplementary).

There exists an EML term of size 27 evaluating to 2. Existential; the direct-search variant has K=19.

```

import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

inductive EMLTerm : Type
| one : EMLTerm
| eml : EMLTerm → EMLTerm → EMLTerm
deriving Repr

noncomputable def EMLTerm.eval : EMLTerm →
| .one => 1
| .eml t u => Real.exp (EMLTerm.eval t) - Real.log (EMLTerm.eval u)

-- The witness term, built bottom-up for clarity
private def t : EMLTerm := .eml .one .one -- eval = e
private def t : EMLTerm := .eml .one t -- eval = e - 1
private def t : EMLTerm := .eml .one t -- eval = e -
↪ log(e-1)
private def t : EMLTerm := .eml t .one -- eval = exp(e -
↪ log(e-1))
private def t : EMLTerm := .eml .one t -- eval = log(e-1)
private def t : EMLTerm := .eml t t -- eval = e - 2
private def t : EMLTerm := .eml t .one -- eval = exp(e-2)
private def witness : EMLTerm := .eml .one t -- eval = 2

private lemma eval_t' : EMLTerm.eval t = Real.exp 1 := by
simp [t, EMLTerm.eval, Real.log_one]

```

```

private lemma eval_t : EMLTerm.eval t = Real.exp 1 - 1 := by
  simp [t, EMLTerm.eval, eval_t', Real.log_exp]

private lemma eval_t : EMLTerm.eval t = Real.exp 1 - Real.log (Real.exp 1 - 1) := by
  simp [t, EMLTerm.eval, eval_t]

private lemma eval_t : EMLTerm.eval t = Real.exp (Real.exp 1 - Real.log (Real.exp 1 - 1)) :=
  ↪ by
  simp [t, EMLTerm.eval, eval_t, Real.log_one]

private lemma eval_t : EMLTerm.eval t = Real.log (Real.exp 1 - 1) := by
  simp [t, EMLTerm.eval, eval_t, Real.log_exp]

private lemma e_minus_one_pos : (0 : ) < Real.exp 1 - 1 := by
  have h0 : Real.exp 0 = 1 := Real.exp_zero
  have h1 : Real.exp 0 < Real.exp 1 := Real.exp_strictMono (by norm_num)
  linarith

private lemma eval_t : EMLTerm.eval t = Real.exp 1 - 2 := by
  simp only [t, EMLTerm.eval, eval_t, eval_t']
  rw [Real.exp_log e_minus_one_pos]
  linarith [Real.log_exp 1]

private lemma eval_t : EMLTerm.eval t = Real.exp (Real.exp 1 - 2) := by
  simp [t, EMLTerm.eval, eval_t, Real.log_one]

private lemma eval_witness : EMLTerm.eval witness = 2 := by
  simp only [witness, EMLTerm.eval, eval_t]
  rw [Real.log_exp]
  ring

theorem emlterm_for_two : t : EMLTerm, EMLTerm.eval t = 2 :=
  witness, eval_witness

end EML

```

033_ emlterm_for_half EMLTerm whose eval is 1/2

Paper section: §3 Results, EML expression catalog (1/2, K=91) • *Status:* complete • *Difficulty:* 4/5

1/2: K = 91 (literal tree in Supplementary).

There exists an EML term of size 91 evaluating to 1/2. Existential; the direct-search variant has K=29.

```

import Mathlib

namespace EML

inductive EMLTerm : Type
| one : EMLTerm
| eml : EMLTerm → EMLTerm → EMLTerm
deriving Repr

noncomputable def EMLTerm.eval : EMLTerm →

```

```

| .one => 1
| .eml t u => Real.exp (EMLTerm.eval t) - Real.log (EMLTerm.eval u)

open EMLTerm

/-- Zero term: evaluates to 0 -/
private def Z : EMLTerm := eml one (eml (eml one one) one)

/-- Log construction: if eval t > 0 then eval (Lg t) = log (eval t) -/
private def Lg (t : EMLTerm) : EMLTerm := eml Z (eml (eml Z t) one)

-- Building blocks
private def e1 : EMLTerm := eml one (eml one one)
private def log_e1 : EMLTerm := Lg e1
private def e2 : EMLTerm := eml log_e1 (eml one one)
private def exp_e2 : EMLTerm := eml e2 one
private def two_ : EMLTerm := eml one exp_e2
private def eml2 : EMLTerm := eml one two_
private def log_eml2 : EMLTerm := Lg eml2
private def neg_log2 : EMLTerm := eml log_eml2 (eml (eml one one) one)
private def half_term : EMLTerm := eml neg_log2 one

-- Evaluation lemmas
private lemma eval_Z : Z.eval = 0 := by
  simp [Z, EMLTerm.eval, Real.log_one, Real.log_exp]

private lemma eval_Lg {t : EMLTerm} (_ : 0 < t.eval) :
  (Lg t).eval = Real.log t.eval := by
  simp only [Lg, EMLTerm.eval, eval_Z, Real.exp_zero, Real.log_exp, Real.log_one, sub_zero]
  ring

private lemma eval_e1 : e1.eval = Real.exp 1 - 1 := by
  simp [e1, EMLTerm.eval, Real.log_one, Real.log_exp]

private lemma exp_one_sub_one_pos : (0 : ) < Real.exp 1 - 1 := by
  linarith [Real.add_one_le_exp (1 : )]

private lemma eval_log_e1 : log_e1.eval = Real.log (Real.exp 1 - 1) := by
  simp only [log_e1]
  rw [eval_Lg (by rw [eval_e1]; exact exp_one_sub_one_pos), eval_e1]

private lemma eval_e2 : e2.eval = Real.exp 1 - 2 := by
  simp only [e2, EMLTerm.eval, eval_log_e1, Real.exp_log exp_one_sub_one_pos,
    Real.log_one, sub_zero, Real.log_exp]
  ring

private lemma eval_exp_e2 : exp_e2.eval = Real.exp (Real.exp 1 - 2) := by
  simp only [exp_e2, EMLTerm.eval, eval_e2, Real.log_one, sub_zero]

private lemma eval_two : two_.eval = 2 := by
  simp only [two_, EMLTerm.eval, eval_exp_e2, Real.log_exp]; ring

private lemma eval_eml2 : eml2.eval = Real.exp 1 - Real.log 2 := by
  simp only [eml2, EMLTerm.eval, eval_two]

private lemma log_two_le_one : Real.log 2 ≤ 1 := by

```

```

rw [show (1:) = Real.log (Real.exp 1) from (Real.log_exp 1).symm]
exact Real.log_le_log (by norm_num) (by linarith [Real.add_one_le_exp (1:)])

private lemma exp_one_sub_log_two_pos : (0 : ) < Real.exp 1 - Real.log 2 := by
  linarith [exp_one_sub_one_pos, log_two_le_one]

private lemma eval_log_eml2 : log_eml2.eval = Real.log (Real.exp 1 - Real.log 2) := by
  simp only [log_eml2]
  rw [eval_Lg (by rw [eval_eml2]; exact exp_one_sub_log_two_pos), eval_eml2]

private lemma eval_neg_log2 : neg_log2.eval = -Real.log 2 := by
  simp only [neg_log2, EMLTerm.eval, eval_log_eml2, Real.log_exp,
    Real.exp_log exp_one_sub_log_two_pos, Real.log_one, sub_zero]
  ring

private lemma eval_half : half_term.eval = 1 / 2 := by
  simp only [half_term, EMLTerm.eval, eval_neg_log2, Real.log_one, sub_zero,
    Real.exp_neg, Real.exp_log (by norm_num : (0:) < 2)]
  norm_num

theorem emlterm_for_half : t : EMLTerm, EMLTerm.eval t = 1/2 :=
  half_term, eval_half

end EML

```

034_emlterm_for_pi · EMLTerm whose eval is

Paper section: §3 Results, EML expression catalog (, K=193) • *Status:* pending • *Difficulty:* 5/5

: K = 193 (literal tree in Supplementary).

There exists an EML term of size 193 evaluating to π . PROBABLE PERMANENT SORRY: transcribing a 193-node tree from the Supplementary by hand is beyond the budget of this auto-formalization pass.

```

import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic
import Mathlib.Analysis.SpecialFunctions.Pow.Real

namespace EML

inductive EMLTerm : Type
| one : EMLTerm
| eml : EMLTerm → EMLTerm → EMLTerm
deriving Repr

def EMLTerm.eval : EMLTerm →
| .one => 1
| .eml t u => Real.exp (EMLTerm.eval t) - Real.log (EMLTerm.eval u)

/-- Existential statement that  $\pi$  is reachable as an EML term.
LIKELY PERMANENT SORRY: a 193-node literal tree is in the paper's Supplementary
and is beyond the budget of this pass to transcribe. -/
theorem emlterm_for_pi : t : EMLTerm, EMLTerm.eval t = Real.pi := by
  sorry

```

```
end EML
```

035__emlterm_for_i · EMLTerm whose eval is i (imaginary unit)

Paper section: §3 Results, EML expression catalog (i, K=131) • *Status:* pending • *Difficulty:* 5/5

i: K = 131 (literal tree in Supplementary).

There exists an EML term (after lifting to) of size 131 evaluating to i. PROBABLE PERMANENT SORRY: requires both a complex variant of EMLTerm and transcription of the 131-node tree.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Complex.Log

namespace EML

/-- Complex-valued EML term grammar (placeholder). -/
inductive EMLTerm : Type
| one : EMLTerm
| eml : EMLTerm → EMLTerm → EMLTerm
deriving Repr

/-- Evaluation of a complex EML term. -/
def EMLTerm.eval : EMLTerm →
| .one => 1
| .eml t u => Complex.exp (EMLTerm.eval t) - Complex.log (EMLTerm.eval u)

/-- Existential: i is reachable. PERMANENT SORRY pending the 131-node tree. -/
theorem emlterm_for_i : t : EMLTerm, EMLTerm.eval t = Complex.I := by
  sorry

end EML
```

036__emlterm_for_neg_x EMLTerm realising the function $-x$

Paper section: §3 Results, EML expression catalog ($-x$, K=57) • *Status:* partial • *Difficulty:* 5/5

$-x$: K = 57 (compiler) / K = 15 (direct search).

There exists a parameterised EML term of size 57 (or 15 in the direct-search variant) whose evaluation at every x equals $-x$. Existential; the formal proof would lift the successor identity (017) to the term level.

Notes: Re-uses EMLTerm from chunk 023. Side conditions on $x \geq 0$, $x \leq -1$ dropped because Real.log is junk-valued. | Aristotle returned COMPLETE_WITH_ERRORS. Original theorem left with sorry. Aristotle claims no EMLTerm of size 15 evaluates to $-x$ (exhaustive search over 109,824 terms) — BUT paper’s tree has K=57 (~28 nodes), beyond Aristotle’s search budget; claim suspect. Aristotle also delivered a verified proof in a richer grammar EMLTerm with `const : → EMLTerm` using Real.log 2 as a parameter. Needs human review.

```
import Mathlib

namespace EML
```



```

inductive EMLTerm : Type
| one : EMLTerm
| var : EMLTerm
| eml : EMLTerm → EMLTerm → EMLTerm
deriving Repr

noncomputable def EMLTerm.eval (x : ) : EMLTerm →
| .one => 1
| .var => x
| .eml t u => Real.exp (EMLTerm.eval x t) - Real.log (EMLTerm.eval x u)

/-!
## Analysis of `emlterm1_for_neg_x`

The theorem as stated with the `EMLTerm` type appears to be unprovable (likely
false). Exhaustive computational search over all 109,824 EML terms of size 15
confirmed that no term evaluates to exactly  $-x$ .

### Why no finite EML term can represent  $-x$ 

For `eml A B` to equal  $-x$ , we need  $\log(\text{eval } B) = \exp(\text{eval } A) + x$ , hence
 $\text{eval } B = \exp(\exp(\text{eval } A) + x)$ . Building  $\exp(c) + x$  (for any constant  $c$ ) as a
sub-term requires either:
1.  $-x$  itself (circular), or
2. A constant like  $\text{Real.log } 2$  that is not in the closure of  $\{0, 1\}$  under
 $\exp$  and  $(a, b) \mapsto \exp(a) - \log(b)$ .

The set of achievable constants  $c$  such that  $c + x$  is EML-representable was
computationally verified to be  $\{0, \pm(e-1), \pm(\exp(e)-e), \dots\}$  - none equal to  $1$ .

### Corrected version

The informal description mentions a "parameterised" EML term. Adding a `const : →`
constructor (yielding `EMLTerm` below) makes the theorem provable, as shown in
`emlterm2_for_neg_x`.
-/

-- Original theorem - left with sorry as it appears to be false for EMLTerm.
theorem emlterm1_for_neg_x :
  t : EMLTerm, x : , EMLTerm.eval x t = -x := by
  sorry

-- =====
-- Verified building blocks for EMLTerm
-- =====

/-- `eml one (eml (eml one one) one)` evaluates to  $0$  for all  $x$ . -/
def zeroTerm : EMLTerm := .eml .one (.eml (.eml .one .one) .one)

lemma eval_zeroTerm (x : ) : zeroTerm.eval x = 0 := by
  simp [zeroTerm, EMLTerm.eval, Real.log_one, Real.log_exp]

/-- `eml zeroTerm (eml var one)` evaluates to  $1 - x$  for all  $x$ . -/
def oneMinusX : EMLTerm := .eml zeroTerm (.eml .var .one)

```

```

lemma eval_oneMinusX (x : ) : oneMinusX.eval x = 1 - x := by
  simp [oneMinusX, EMLTerm.eval, zeroTerm, Real.log_one, Real.log_exp, Real.exp_zero]

-- =====
-- Corrected (parameterised) EML type and proof
-- =====

/-- Extended EML term type with a `const` constructor for real-valued parameters. -/
inductive EMLTerm : Type
| const : → EMLTerm
| var : EMLTerm
| eml : EMLTerm → EMLTerm → EMLTerm

noncomputable def EMLTerm.eval (x : ) : EMLTerm →
| .const c => c
| .var => x
| .eml t u => Real.exp (EMLTerm.eval x t) - Real.log (EMLTerm.eval x u)

/-- There exists a parameterised EML term whose evaluation at every `x` equals `-x`.

**Construction** (using the parameter `Real.log 2`):

| Sub-term | Evaluates to |
|---|---|
| `onemx` := eml (const 0) (eml var (const 1))` | `1 - x` |
| `onepx` := eml (const (log 2)) (eml onemx (const 1))` | `1 + x` |
| `negx` := eml (const 0) (eml onepx (const 1))` | `-x` |

**Identity chain**:
- `exp(0) - log(1) = 1` and `exp(x) - log(1) = exp(x)`, so `onemx = 1 - x`.
- `exp(log 2) - log(exp(1-x)) = 2 - (1-x) = 1 + x`, so `onepx = 1 + x`.
- `exp(0) - log(exp(1+x)) = 1 - (1+x) = -x`, so `negx = -x`. -/
theorem emlterm2_for_neg_x :
  t : EMLTerm, x : , EMLTerm.eval x t = -x := by
  let onemx : EMLTerm := .eml (.const 0) (.eml .var (.const 1))
  let onepx : EMLTerm := .eml (.const (Real.log 2)) (.eml onemx (.const 1))
  let negx : EMLTerm := .eml (.const 0) (.eml onepx (.const 1))
  exact negx, fun x => by
    simp only [negx, onepx, onemx, EMLTerm.eval]
    simp [Real.log_one, Real.log_exp, Real.exp_zero,
      Real.exp_log (by positivity : (0 : ) < 2)]
    ring
end EML

```

037_ emlterm_for_inv_x EMLTerm realising $1/x$ (for $x > 0$)

Paper section: §3 Results, EML expression catalog ($1/x$, $K=65$) • *Status:* complete • *Difficulty:* 5/5

$1/x$: $K = 65$ (compiler) / $K = 15$ (direct search).

There exists a parameterised EML term of size 65 (or 15 direct-search) whose evaluation equals $1/x$ for every nonzero x . Existential; the $x \neq 0$ constraint is semantic — Real has junk values at 0.

```

import Mathlib

namespace EML

inductive EMLTerm : Type
| one : EMLTerm
| var : EMLTerm
| eml : EMLTerm → EMLTerm → EMLTerm
deriving Repr

noncomputable def EMLTerm.eval (x : ) : EMLTerm →
| .one => 1
| .var => x
| .eml t u => Real.exp (EMLTerm.eval x t) - Real.log (EMLTerm.eval x u)

-- The key subterms
/-- log(x) for x > 0 -/
noncomputable def logTerm : EMLTerm := .eml .one (.eml (.eml .one .var) .one)

/-- x - log(x) for x > 0 -/
noncomputable def xMinusLogTerm : EMLTerm := .eml logTerm .var

/-- log(x - log(x)) for x > 0 -/
noncomputable def logXMinusLogTerm : EMLTerm := .eml .one (.eml (.eml .one xMinusLogTerm)
  ↪ .one)

/-- -log(x) for x > 0 -/
noncomputable def negLogTerm : EMLTerm := .eml logXMinusLogTerm (.eml .var .one)

/-- 1/x for x > 0 -/
noncomputable def invTerm : EMLTerm := .eml negLogTerm .one

/-
Helper: x - log(x) > 0 for x > 0
-/
lemma sub_log_pos {x : } (hx : 0 < x) : 0 < x - Real.log x := by
  linarith [ Real.log_le_sub_one_of_pos hx ]

-- Step 1: logTerm evaluates to log(x)
lemma eval_logTerm {x : } (_hx : 0 < x) :
  EMLTerm.eval x logTerm = Real.log x := by
  simp only [logTerm, EMLTerm.eval, Real.log_one, sub_zero, Real.log_exp]
  ring

-- Step 2: xMinusLogTerm evaluates to x - log(x)
lemma eval_xMinusLogTerm {x : } (hx : 0 < x) :
  EMLTerm.eval x xMinusLogTerm = x - Real.log x := by
  simp only [xMinusLogTerm, EMLTerm.eval, eval_logTerm hx, Real.exp_log hx]

-- Step 3: logXMinusLogTerm evaluates to log(x - log(x))
lemma eval_logXMinusLogTerm {x : } (hx : 0 < x) :
  EMLTerm.eval x logXMinusLogTerm = Real.log (x - Real.log x) := by
  simp only [logXMinusLogTerm, EMLTerm.eval, eval_xMinusLogTerm hx, Real.log_one, sub_zero,
    Real.log_exp]
  ring

```

```

-- Step 4: negLogTerm evaluates to -log(x)
lemma eval_negLogTerm {x : } (hx : 0 < x) :
  EMLTerm.eval x negLogTerm = -Real.log x := by
  simp only [negLogTerm, EMLTerm.eval, eval_logXMinusLogTerm hx,
    Real.exp_log (sub_log_pos hx), Real.log_one, sub_zero, Real.log_exp]
  ring

-- Step 5: invTerm evaluates to 1/x
lemma eval_invTerm {x : } (hx : 0 < x) :
  EMLTerm.eval x invTerm = 1 / x := by
  simp only [invTerm, EMLTerm.eval, eval_negLogTerm hx, Real.log_one, sub_zero]
  rw [Real.exp_neg, Real.exp_log hx, one_div]

theorem emlterm1_for_inv_x_pos :
  t : EMLTerm, x : , 0 < x → EMLTerm.eval x t = 1 / x := by
  exact invTerm, fun x hx => eval_invTerm hx

end EML

```

038_emlterm_for_sq_x EMLTerm realising x^2 (for $x > 0$)

Paper section: §3 Results, EML expression catalog (x^2 , $K=75$) • *Status:* complete • *Difficulty:* 5/5

x^2 : $K = 75$ (compiler) / $K = 17$ (direct search).

There exists a parameterised EML term of size 75 (or 17 direct-search) whose evaluation equals x^2 for every x . Existential.

```

import Mathlib

namespace EML

inductive EMLTerm : Type
| one : EMLTerm
| var : EMLTerm
| eml : EMLTerm → EMLTerm → EMLTerm
deriving Repr

noncomputable def EMLTerm.eval (x : ) : EMLTerm →
| .one => 1
| .var => x
| .eml t u => Real.exp (EMLTerm.eval x t) - Real.log (EMLTerm.eval x u)

-- Building blocks
private def zeroTerm : EMLTerm := .eml .one (.eml (.eml .one .one) .one)
private def logTerm : EMLTerm := .eml zeroTerm (.eml (.eml zeroTerm .var) .one)
private def xMinusLogTerm : EMLTerm := .eml logTerm .var
private def logXMinusLogTerm : EMLTerm :=
  .eml zeroTerm (.eml (.eml zeroTerm xMinusLogTerm) .one)
private def xMinus2LogTerm : EMLTerm :=
  .eml logXMinusLogTerm (.eml logTerm .one)
private def twoLogTerm : EMLTerm :=
  .eml logTerm (.eml xMinus2LogTerm .one)
private def sqTerm : EMLTerm := .eml twoLogTerm .one

```

```

/-
Helper:  $x - \log x > 0$  for  $x > 0$ 
-/
private lemma x_minus_log_pos (x : ℝ) (hx : 0 < x) : 0 < x - Real.log x := by
  linarith [ Real.log_le_sub_one_of_pos hx ]

/-
Step-by-step evaluation lemmas
-/
private lemma eval_zeroTerm (x : ℝ) : zeroTerm.eval x = 0 := by
  simp [zeroTerm, EMLTerm.eval]

private lemma eval_logTerm (x : ℝ) (hx : 0 < x) : logTerm.eval x = Real.log x := by
  unfold logTerm; simp +decide [ *, EMLTerm.eval ] ;

private lemma eval_xMinusLogTerm (x : ℝ) (hx : 0 < x) :
  xMinusLogTerm.eval x = x - Real.log x := by
  convert congr_arg ( · - · ) ( Real.exp_log hx ) rfl using 1;
  convert congr_arg ( · - · ) ( congr_arg Real.exp ( eval_logTerm x hx ) ) rfl using 1

private lemma eval_logXMinusLogTerm (x : ℝ) (hx : 0 < x) :
  logXMinusLogTerm.eval x = Real.log (x - Real.log x) := by
  unfold logXMinusLogTerm;
  -- We'll use the fact that $zeroTerm.eval x = 0$ and $xMinusLogTerm.eval x = x - \log
  ↪ x$.
  have h_eval : zeroTerm.eval x = 0 xMinusLogTerm.eval x = x - Real.log x := by
    exact eval_zeroTerm x, eval_xMinusLogTerm x hx
  simp [h_eval, EMLTerm.eval]

private lemma eval_xMinus2LogTerm (x : ℝ) (hx : 0 < x) :
  xMinus2LogTerm.eval x = x - 2 * Real.log x := by
  convert congr_arg ( · - · ) ( Real.exp_log ( x_minus_log_pos x hx ) ) ( Real.log_exp (
  ↪ Real.log x ) ) using 1;
  · rw [ show xMinus2LogTerm = .eml logXMinusLogTerm (.eml logTerm .one) from rfl, show
  ↪ logXMinusLogTerm = .eml zeroTerm (.eml (.eml zeroTerm xMinusLogTerm) .one) from rfl, show
  ↪ logTerm = .eml zeroTerm (.eml (.eml zeroTerm .var) .one) from rfl, show zeroTerm = .eml
  ↪ .one (.eml (.eml .one .one) .one) from rfl ] ; simp +decide [ EMLTerm.eval ] ;
  · rw [ eval_xMinusLogTerm x hx ];
  · ring

private lemma eval_twoLogTerm (x : ℝ) (hx : 0 < x) :
  twoLogTerm.eval x = 2 * Real.log x := by
  unfold twoLogTerm;
  -- Apply the definitions of `logTerm` and `xMinus2LogTerm` to simplify the expression.
  simp [logTerm, xMinus2LogTerm, EMLTerm.eval];
  rw [ eval_logXMinusLogTerm x hx, Real.exp_log ( x_minus_log_pos x hx ) ] ; ring;
  rw [ Real.exp_log hx, sub_self, zero_add ]

private lemma eval_sqTerm (x : ℝ) (hx : 0 < x) :
  sqTerm.eval x = x ^ 2 := by
  -- Simplify $sqTerm$ using the results of $twoLogTerm$ and basic properties of
  ↪ exponentiation.
  have h_exp_simplified :
    Real.exp (2 * Real.log x) = x ^ 2 := by
    rw [ mul_comm, Real.exp_mul, Real.exp_log ] <;> norm_cast;
  convert h_exp_simplified using 1

```

```

    unfold EMLTerm.eval
    simp [sqTerm];
    rw [ eval_twoLogTerm x hx, show EMLTerm.eval x EMLTerm.one = 1 from rfl,
    ↪ Real.log_one, sub_zero ]

theorem emlterm1_for_sq_x_pos :
  t : EMLTerm, x : ℝ, 0 < x → EMLTerm.eval x t = x ^ 2 := by
  exact sqTerm, eval_sqTerm

end EML

```

039_emlterm_for_sqrt_x · EMLTerm realising the function $\sqrt{x}$

Paper section: §3 Results, EML expression catalog ($\sqrt{x}$, K=139) • *Status:* pending • *Difficulty:* 5/5

$\sqrt{x}$: K = 139 (compiler) / K > 43 (direct search).

There exists a parameterised EML term of size 139 whose evaluation equals $\sqrt{x}$ for $x \geq 0$. PROBABLE PERMANENT SORRY: 139-node literal tree beyond the manual-transcription budget.

```

import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic
import Mathlib.Analysis.SpecialFunctions.Pow.Real

namespace EML

inductive EMLTerm : Type
| one : EMLTerm
| var : EMLTerm
| eml : EMLTerm → EMLTerm → EMLTerm
deriving Repr

def EMLTerm.eval (x : ℝ) : EMLTerm → ℝ
| .one => 1
| .var => x
| .eml t u => Real.exp (EMLTerm.eval x t) - Real.log (EMLTerm.eval x u)

theorem emlterm1_for_sqrt_x :
  t : EMLTerm, x : ℝ, 0 ≤ x → EMLTerm.eval x t = Real.sqrt x := by
  sorry

end EML

```

040_emlterm_for_add_xy · EMLTerm realising $x + y$

Paper section: §3 Results, EML expression catalog ($x + y$, K=27) • *Status:* complete • *Difficulty:* 5/5

$x + y$: K = 27 (compiler) / K = 19 (direct search).

There exists a two-variable EML term of size 27 whose evaluation at (x,y) equals $x + y$. Requires a 2-variable grammar EMLTerm (with .varX, .varY leaves) and an evaluation $\text{eval} : \text{EMLTerm} \rightarrow \mathbb{R}$.

```

import Mathlib

namespace EML

/-- Two-variable EML term grammar. -/
inductive EMLTerm : Type
| one : EMLTerm
| varX : EMLTerm
| varY : EMLTerm
| eml : EMLTerm → EMLTerm → EMLTerm
deriving Repr

/-- Evaluation of a two-variable EML term at (x, y). -/
noncomputable def EMLTerm.eval (x y : ℝ) : EMLTerm → ℝ
| .one => 1
| .varX => x
| .varY => y
| .eml t u => Real.exp (EMLTerm.eval x y t) - Real.log (EMLTerm.eval x y u)

/-
exp(x) - x is always positive.
-/
lemma exp_sub_self_pos (x : ℝ) : 0 < Real.exp x - x := by
  linarith [Real.add_one_le_exp x]

theorem emlterm2_for_add :
  t : EMLTerm, x y : ℝ, EMLTerm.eval x y t = x + y := by
  refine .eml
  (.eml .one (.eml (.eml .one (.eml .varX .one)) .one))
  (.eml
    (.eml (.eml .one (.eml (.eml .one (.eml .varX (.eml .varX .one))) .one))
      (.eml .varY .one))
    .one), ?_
  intro x y
  simp only [EMLTerm.eval, Real.log_one, sub_zero, Real.log_exp]
  have h1 : Real.exp 1 - (Real.exp 1 - x) = x := by ring
  have h2 : Real.exp 1 - (Real.exp 1 - Real.log (Real.exp x - x)) =
    Real.log (Real.exp x - x) := by ring
  rw [h1, h2, Real.exp_log (exp_sub_self_pos x)]
  ring
end EML

```

041_emlterm_for_mul_xy EMLTerm realising $x \cdot y$

Paper section: §3 Results, EML expression catalog ($x \times y$, $K=41$) • *Status:* complete • *Difficulty:* 5/5

$x \times y$: $K = 41$ (compiler) / $K = 17$ (direct search).

There exists a two-variable EML term of size 41 whose evaluation equals $x \cdot y$ for $x, y > 0$ (Identity 1). Existential.

```

import Mathlib

namespace EML

```

```

inductive EMLTerm : Type
| one : EMLTerm
| varX : EMLTerm
| varY : EMLTerm
| eml : EMLTerm → EMLTerm → EMLTerm
deriving Repr

noncomputable def EMLTerm.eval (x y : ℝ) : EMLTerm → ℝ
| .one => 1
| .varX => x
| .varY => y
| .eml t u => Real.exp (EMLTerm.eval x y t) - Real.log (EMLTerm.eval x y u)

/-
For x > 0, x - log x > 0
-/
lemma sub_log_pos {x : ℝ} (hx : 0 < x) : 0 < x - Real.log x := by
  linarith [Real.log_le_sub_one_of_pos hx]

theorem emlterm2_for_mul :
  t : EMLTerm, x y : ℝ, 0 < x → 0 < y → EMLTerm.eval x y t = x * y := by
  refine ?_, fun x y hx hy => ?_
  · exact .eml (.eml (.eml .one (.eml (.eml .one .varX) .one))
    (.eml (.eml (.eml .one (.eml (.eml .one
      (.eml (.eml .one (.eml (.eml .one .varX) .one))
        (.eml (.eml .one (.eml (.eml .one .varX) .one)) .one))) .one)) .varY) .one)) .one
  · simp only [EMLTerm.eval, Real.log_one, sub_zero, Real.log_exp]
    -- Goal has e - (e - log x) patterns where e = exp 1
    set e := Real.exp 1
    -- Step 1: simplify e - (e - log x) to log x
    have h1 : e - (e - Real.log x) = Real.log x := by ring
    rw [h1]
    -- Step 2: exp(log x) = x
    rw [Real.exp_log hx]
    -- Step 3: simplify e - (e - log(x - log x)) to log(x - log x)
    have h3 : e - (e - Real.log (x - Real.log x)) = Real.log (x - Real.log x) := by ring
    rw [h3]
    -- Step 4: exp(log(x - log x)) = x - log x
    rw [Real.exp_log (sub_log_pos hx)]
    -- Step 5 & 6: exp(x - (x - log x - log y)) = exp(log x + log y) = x * y
    have h5 : x - (x - Real.log x - Real.log y) = Real.log x + Real.log y := by ring
    rw [h5, Real.exp_add, Real.exp_log hx, Real.exp_log hy]

end EML

```

042_emlterm_for_pow_xy EMLTerm realising x^y (for $0 < x$ and $0 < y$)

Paper section: §3 Results, EML expression catalog (x^y , K=49) • *Status:* complete • *Difficulty:* 5/5

x^y : K = 49 (compiler) / K = 25 (direct search).

There exists a two-variable EML term of size 49 (or 25 direct-search) whose evaluation equals $x^y = \exp(y \cdot \ln x)$ for $x > 0$. Existential.


```

import Mathlib

namespace EML

inductive EMLTerm : Type
| one : EMLTerm
| varX : EMLTerm
| varY : EMLTerm
| eml : EMLTerm → EMLTerm → EMLTerm
deriving Repr

noncomputable def EMLTerm.eval (x y : ) : EMLTerm →
| .one => 1
| .varX => x
| .varY => y
| .eml t u => Real.exp (EMLTerm.eval x y t) - Real.log (EMLTerm.eval x y u)

-- Helper definitions for building the term
private def Z : EMLTerm := .eml .one (.eml (.eml .one .one) .one)
private def LOG (a : EMLTerm) : EMLTerm := .eml Z (.eml (.eml Z a) .one)
private def NEG_LOG (v : EMLTerm) (raw : EMLTerm) : EMLTerm :=
.eml (LOG (.eml v raw)) (.eml raw .one)

/-- EML term that computes  $x^y$  for  $x > 0, y > 0$ .
Key identity:  $y * \log(x) = y * (1/x + \log(x)) - y/x$ ,
where both  $1/x + \log(x) > 0$  and  $1/x > 0$  for  $x > 0$ . -/
noncomputable def pow_term : EMLTerm :=
let logx := LOG .varX
let logy := LOG .varY
let neg_logx := NEG_LOG logx .varX
let neg_logy := NEG_LOG logy .varY
let inv_y_plus_logy := EMLTerm.eml neg_logy (.eml neg_logy .one)
let log_inv_y_plus_logy := LOG inv_y_plus_logy
let inv_x_plus_logx := EMLTerm.eml neg_logx (.eml neg_logx .one)
let log_inv_x_plus_logx := LOG inv_x_plus_logx
let A_arg := EMLTerm.eml log_inv_y_plus_logy
(.eml (.eml neg_logy (.eml log_inv_x_plus_logx .one)) .one)
let B_arg := EMLTerm.eml log_inv_y_plus_logy
(.eml (.eml neg_logy (.eml neg_logx .one)) .one)
let A := EMLTerm.eml A_arg .one
let B := EMLTerm.eml B_arg .one
let y_logx := EMLTerm.eml (LOG A) (.eml B .one)
EMLTerm.eml y_logx .one

/-
Auxiliary lemmas
-/
private lemma inv_add_log_pos {a : } (ha : 0 < a) : 0 < a-1 + Real.log a := by
nlinarith [ inv_pos.2 ha, mul_inv_cancel ha.ne', Real.log_inv a
↪ Real.log_le_sub_one_of_pos ( inv_pos.2 ha ) ]

private lemma eval_Z (x y : ) : EMLTerm.eval x y Z = 0 := by
unfold Z;
-- By definition of  $Z$ , we know that  $Z = \exp(1) - \log(\exp(\exp(1) - \log(1)))$ .
simp [EMLTerm.eval]

```

```

private lemma eval_LOG (x y : ) (a : EMLTerm) (ha : 0 < EMLTerm.eval x y a) :
  EMLTerm.eval x y (LOG a) = Real.log (EMLTerm.eval x y a) := by
  unfold LOG;
  -- By definition of `LOG`, we know that `EMLTerm.eval x y (Z.eml ((Z.eml a).eml
  ↪ EMLTerm.one)) = Real.log (EMLTerm.eval x y a)`.
  simp [EMLTerm.eval]

private lemma eval_NEG_LOG (x y : ) (v raw : EMLTerm)
  (hraw : 0 < EMLTerm.eval x y raw)
  (hv : EMLTerm.eval x y v = Real.log (EMLTerm.eval x y raw))
  (_hd : 0 < EMLTerm.eval x y raw - EMLTerm.eval x y v) :
  EMLTerm.eval x y (NEG_LOG v raw) = -(EMLTerm.eval x y v) := by
  unfold NEG_LOG;
  unfold LOG; norm_num [ EMLTerm.eval ] ;
  rw [ Real.exp_log ] <;> norm_num [ hv ] ; linarith [ Real.exp_log hraw ] ;
  linarith [ Real.add_one_le_exp ( Real.log ( EMLTerm.eval x y raw ) ) ]

private lemma eval_pow_term_eq (x y : ) (hx : 0 < x) (hy : 0 < y) :
  EMLTerm.eval x y pow_term = Real.exp (y * Real.log x) := by
  -- Let's simplify the expression step by step.
  have h1 : EMLTerm.eval x y (LOG .varX) = Real.log x := by
    exact eval_LOG x y _ hx
  have h2 : EMLTerm.eval x y (LOG .varY) = Real.log y := by
    exact eval_LOG x y _ hy
  have h3 : EMLTerm.eval x y (NEG_LOG (LOG .varX) .varX) = -Real.log x := by
    rw [ ← h1 ] ;
    apply_rules [ eval_NEG_LOG ] ;
    exact sub_pos_of_lt ( by linarith [ Real.log_le_sub_one_of_pos hx, show EMLTerm.eval x y
  ↪ EMLTerm.varX = x from rfl ] )
  have h4 : EMLTerm.eval x y (NEG_LOG (LOG .varY) .varY) = -Real.log y := by
    rw [ ← h2 ] ;
    apply_rules [ eval_NEG_LOG ] ;
    exact sub_pos_of_lt ( by linarith [ Real.log_le_sub_one_of_pos hy, show EMLTerm.eval x y
  ↪ EMLTerm.varY = y from rfl ] ) ;
  unfold pow_term; simp +decide [ *, EMLTerm.eval ] ; ring;
  simp +decide [ *, EMLTerm.eval, LOG ] at * ;
  norm_num [ Real.exp_add, Real.exp_sub, Real.exp_neg, Real.exp_log hx, Real.exp_log hy ] ;
  ↪ ring;
  rw [ Real.exp_log ( by linarith [ inv_pos.2 hy, Real.log_inv y Real.log_le_sub_one_of_pos
  ↪ ( inv_pos.2 hy ) ] ), Real.exp_log ( by linarith [ inv_pos.2 hx, Real.log_inv x
  ↪ Real.log_le_sub_one_of_pos ( inv_pos.2 hx ) ] ) ] ; norm_num [ Real.exp_ne_zero, hx.ne',
  ↪ hy.ne' ] ; ring;
  norm_num [ Real.exp_add, Real.exp_log hy, mul_assoc, mul_comm, mul_left_comm, ne_of_gt (
  ↪ Real.exp_pos _ ) ]

theorem emlterm2_for_pow_pos :
  t : EMLTerm, x y : , 0 < x → 0 < y → EMLTerm.eval x y t = x ^ y := by
  exact pow_term, fun x y hx hy => by
    rw [eval_pow_term_eq x y hx hy]
    rw [Real.rpow_def_of_pos hx]
    ring_nf

end EML

```

043__master_formula_param_count · Master-formula parameter count at level n

Paper section: §4.3 Master formula - symbolic regression • *Status:* pending • *Difficulty:* 2/5

Level-n EML master formula has $5 \times 2^n - 6$ parameters total.

The level-n master formula has $5 \cdot 2^n - 6$ parameters. We define `parametrCount n := 5 · 2n − 6` and check small values (n=1: 4, n=2: 14, n=3: 34).

```
import Mathlib.Algebra.GroupPower.Basic

namespace EML

/-- Total parameter count of the level-n EML master formula:
`5 · 2^n - 6` (Section 4.3). -/
def masterParamCount (n : ℕ) : ℕ := 5 * 2 ^ n - 6

example : masterParamCount 1 = 4 := by sorry
example : masterParamCount 2 = 14 := by sorry
example : masterParamCount 3 = 34 := by sorry

end EML
```

044__emlterm_count_catalan · Count of EMLTerms equals the Catalan number

Paper section: §4.2 Elementary functions as binary trees ('Catalan structures') • *Status:* pending • *Difficulty:* 4/5

Context-free language; isomorphic to full binary trees / Catalan structures.

The number of full binary trees with n leaves is the Catalan number C_{n-1} . By induction, the count of EMLTerms of size $2k+1$ is C_k . Mathlib has `Nat.catalan`.

```
import Mathlib.Combinatorics.Catalan

namespace EML

inductive EMLTerm : Type
| one : EMLTerm
| eml : EMLTerm → EMLTerm → EMLTerm
deriving Repr

def EMLTerm.size : EMLTerm → ℕ
| .one => 1
| .eml t u => 1 + EMLTerm.size t + EMLTerm.size u

/-- Number of EML terms of size `2k + 1` equals the Catalan number `C`.
The set of EMLTerms of bounded size is finite; we phrase the count via
a finset cardinality (Fintype instance left as `sorry` machinery). -/
theorem emlterm_count_catalan (k : ℕ) :
  (S : Finset EMLTerm), ( t ∈ S, EMLTerm.size t = 2 * k + 1 )
  ( t : EMLTerm, EMLTerm.size t = 2 * k + 1 → t ∈ S )
  S.card = Nat.catalan k := by
  sorry

end EML
```

045_main_completeness_stub · Main completeness theorem — stub

Paper section: §3 Results, abstract claim of universality • *Status:* pending • *Difficulty:* 5/5

EML + 1 generates all standard scientific calculator operations.

Umbrella statement: for each of the 36 primitives in Table 1 there exists a (parameterised) EML term realising it. Left as **sorry** until every sub-case (chunks 030–042) is closed.

```
namespace EML

/-- Main completeness umbrella (placeholder). Holds vacuously as `True` until
each constructive sub-case (chunks 030-042) is settled. -/
theorem main_completeness_stub : True := by
  sorry

end EML
```